

TECHNICAL PRESENTATION

SESSION TRACKING TECHNIQUES

HttpSession, Cookies, and URL Rewriting: Advantages and Limitations

Focus Areas: Session Persistence | Security Concerns | Use-Case Comparison

NAME: Y.Sai Prasanth Varma

REGISTRATION NUMBER: 192411191

COURSE CODE:CSA4308

GUIDED BY: Dr.Rajagopal

OVERVIEW

Introduction: Why Session Tracking?

HTTP is stateless — each request is independent. Session tracking lets an application recognize a returning client across requests, to maintain things like login status or cart contents.

HttpSession	A server-side object that stores session state, keyed by a session identifier.
--------------------	--

Cookies	Client-side data stores, sent automatically with every request to the domain.
----------------	---

URL Rewriting	The session ID is encoded directly into the page's links, instead of a cookie.
----------------------	--

Session Persistence

Mechanism	How State Persists	Limitation
HttpSession	Held in server memory, referenced by a session ID	Lost on restart unless replicated; needs a shared store to scale
Cookies	Stored in the browser, per Expires/Max-Age	Lost if the user clears data or disables cookies
URL Rewriting	Carried in every rewritten link on the page	Lost if a bookmarked or shared URL is reused

FOCUS AREA 2

Security Concerns

HttpSession

- Session hijacking if the ID is exposed (sniffing, XSS)
- Session fixation before authentication
- Session store is a high-value target

Cookies

- Readable by scripts without the HttpOnly flag
- Vulnerable to CSRF without SameSite
- Exposed in transit without the Secure flag

URL Rewriting

- Session ID visible in history and server logs
- Leaked via the Referer header
- Easily shared or bookmarked by accident

Use-Case Comparison

Mechanism	Advantage	Best Suited For
HttpSession	Full server-side control; not exposed to the client	Sensitive state — banking, enterprise dashboards
Cookies	Lightweight; no server lookup needed for simple data	Preferences, "remember me" login, tracking
URL Rewriting	Works even when cookies are disabled	Fallback when cookie support can't be assumed

SUMMARY

Conclusion

No single mechanism is universally best — each is a trade-off between server load, client dependency, and exposure risk.

- **HttpSession:** gives the strongest server-side control, at the cost of memory and scaling overhead.
 - **Cookies:** are the most common way to carry a session ID, but depend on the client accepting them.
 - **URL Rewriting:** is mainly a fallback, since it exposes the session ID in URLs and logs.
-

Thank
you

